

Solitito — Project Summary

A real-time guitar practice and recognition system

Updated 4 October 2026 — take7 integration, application version 0.5.7

This document describes version 0.5.7 with the take7 model and the experiments that led to it. Earlier take6 measurements are labelled as historical; they are not scores for the new Rise detector. The model's take number is separate from the application's release version.

1. System overview

Solitito is a real-time guitar trainer implemented in Rust. The program takes a signal from a microphone or audio interface, recognises the material being played, and guides the user through jazz standards, intervals, scales, arpeggios, interval formulas and the layout of the neck.

Recognition uses a single take7 ONNX containing the chord network and a separate Rise onset branch. All processing — DSP, inference and the user interface — is carried out locally on the CPU, without a network connection and without external services.

Six modes of operation are provided:

- **Chords** — complete jazz standards. Green indicates an exact match, yellow a triad or a common substitution, red a chord that was recognised but at a signal level too low to confirm.
- **Intervals** — chord tones played individually, with a selectable set of degrees to practise.
- **Scales** — sequential traversal of the notes defined by a scale.
- **Arpeggios** — chord tones in sequence over a given progression.
- **Fretboard** — a region of the neck is drawn at random, comprising a set of strings and a span of four frets, and then held; the user is asked for successive notes lying within it. The mode serves to learn the positions of the notes within a single hand position.
- **Formulas** — a set of intervals drawn over a root and played in any order, with the option of planting that same set on a chord or carrying it across the chords of a standard. The mode is described in 8.11.

Work on the project began in December 2025. This document presents the system architecture, the course of the work, and the design decisions together with their justification.

2. Methodology

The project adopted the principle that every change requires justification by measurement rather than by hypothesis. In practice this meant developing a set of **probes** — scripts that answer a single question at low computational cost, without requiring retraining.

probe	question addressed
<code>verify_annotations.py</code>	do the labels describe the audio content?
<code>probe_root.py</code>	how often does the labelled root actually sound within the window?
<code>probe_sources.py</code>	which GuitarSet chord annotation is usable?
<code>probe_quality.py</code>	from which source should chord quality be derived?
<code>inspect_jams.py</code>	what do the JAMS files actually contain?
<code>latency_material.py</code>	generates plucks whose onsets are known exactly, as a yardstick
<code>latency_ground_truth.py</code>	extracts onsets and pitches from a REAL recording, for the same measurement
<code>latency_stats.py</code>	how late does the application learn what was played, and how often wrongly?
<code>latency_rules.py</code>	what would each crediting rule cost on that recording?

The first five address the data and the model; the remaining four form a chain and are used together. Material is prepared — either synthesised, with onsets known by construction, or taken from a recording of real playing — the file is then passed through the application's own feature path with `./solitito --probe file.wav --step 1`, and the resulting listing is read by the last two scripts. `latency_stats.py` separates the three answers the application can hold: the model's pitch head, the single-frame estimate and the onset head. `latency_rules.py` replays the crediting rules over the same listing and reports, for each, how many credits it would hand out that nobody played and how many notes it would miss altogether. The table in 8.11 comes from that script.

Two further tools are not probes but belong to the same set: `gen_weights.py`, which produces the sparse CQT kernel shared by the trainer and the application, and `gp5_to_arpeggio.py`, which converts a Guitar Pro file into the degree notation the Arpeggios mode reads. `hf_cleanup.py` clears the checkpoint repository before a run begins from scratch.

The methodology proved effective on repeated occasions. Its converse should also be noted: **hypotheses formulated prior to measurement proved to be wrong in a systematic manner.** These cases are catalogued in Chapter 9.

3. The synthetic dataset

The script `dataset_generator_v2.py` produces, in a single pass, a Guitar Pro file **together with** a complete set of annotations.

3.1. Contents

394 blocks of 6 seconds each (3 bars at 120 BPM), comprising:

- 12 roots $\times$ {maj, min, maj7, dom7, min7, m7b5, dim7, sus4, aug} in several positions on the fretboard,
- all 96 single notes (6 strings $\times$ 16 frets).

Block structure: the first bar constitutes the attack, the second the sustain (tie), and the third silence. The annotation covers the interval `[start + 0.05 s, +3.2 s]`, that is the attack together with the sustain, excluding the decay tail.

3.2. Verification of chord shapes

Before generation begins, the script checks **each of the 21 movable shapes at every fret**, verifying that the shape genuinely produces the declared intervals. An error in the shape table halts generation rather than propagating into the dataset.

3.3. Rendering

The guitar track is exported as a DI signal and rendered in a DAW using [NAM](#) in two variants:

- `synth_dataset_clean.wav` — Fender Deluxe Reverb, clean tone,
- `synth_dataset_eob.wav` — at the edge of breakup.

The recommended sample rate is **48 kHz**, for two independent reasons. NAM operates natively at 48 kHz, so at 44.1 kHz the plugin performs internal resampling. Furthermore, decimation to the 16 kHz at which the model operates is exact at 48 kHz ($48000/16000 = 3$) and is not exact at 44.1 kHz (2.75625).

3.4. Calibration and verification

The `--calibrate <wav>` mode determines the position of the first attack, in case the DAW has added silence at the beginning of the file. PCM 16/24/32-bit and floating-point 32/64-bit formats are supported, in mono and stereo, by means of a self-contained WAV reader with no external dependencies.

The script `verify_annotations.py` compares the label against the **actual audio content**: the dominant pitch class within the window is set against the root indicated by the label. Its only dependency is numpy.

Reference values obtained from the v2 render:

	clean	eob	random baseline
top1	87%	77%	~8%
top3	100%	98%	~25%

The top3 figure is the meaningful one, since within a chord the root is frequently quieter than the third or the fifth. A value of $\text{top3} > 75\%$ qualifies the dataset for training; $\text{top3} \approx 25\%$ indicates that the labels do not describe the signal.

The script additionally verifies the block period derived from the energy envelope, together with the offset between the start of the annotation and the attack. The two tests are **complementary**: an annotation shifted by an integer multiple of the block length coincides with the attack of an adjacent block and remains undetectable by the timing test. Only a comparison of the labels against the signal content will reveal it.

An overriding principle was adopted: **the generator writes labels directly, and an independent script verifies their agreement with the audio**. No processing step reconstructs labels from the signal.

4. The GuitarSet dataset

GuitarSet comprises 360 recordings with annotations in JAMS format, captured with a hexaphonic pickup. It is the only source of material from a real instrument used in the project. Bringing it into a usable form required four training runs.

Four properties of the dataset are described below, the omission of which reduces model accuracy.

For Rise, both solo and comp recordings supply note-start labels. Their string-resolved annotations are supervision only: the model hears ordinary mono audio and does not require a hexaphonic pickup in the application. The chord-label problems discussed below do not justify discarding solo note annotations.

4.1. Half of the dataset contains no chords

Every excerpt was recorded twice: as `_comp` (accompaniment) and as `_solo` (monophonic improvisation). **The chord annotation is identical in both cases** — it describes the progression over which the performer improvised.

Training the chord heads on solo files amounts to teaching the model that a single note constitutes a complete jazz chord. The solo material comprises 180 of the 360 files.

Filtering these files moved the `Exact` figure from **44.8% to 82.3%** in a single run.

The **pitch** targets derived from solo files remain entirely correct — this is genuine monophonic playing with accurate note annotations, and therefore material well suited to a single-note detector. The trainer consequently retains the pitch loss on solo windows and masks the root and quality only (`GUIARSET_SOLO_MODE = "mask_chord"`).

An indirect consequence should be noted. The `probe_root.py` probe initially determined root audibility across all 360 files, yielding a ceiling of 64.1%. On this basis it was concluded that jazz performers employ rootless voicings and that the chord name is not a function of the signal. That conclusion proved incorrect: once the solo material was filtered out, the root sounds in **97%** of accompaniment windows.

4.2. Chord annotations exist in two variants

Every file carries an `instructed` annotation (the chord as written) and a `performed` annotation (a transcription of the performance). The number of segments is identical; the distribution of labels is not:

quality	instructed	performed	difference
maj	2640	2106	-534
min	960	460	-500
min7	0	360	+360
maj7	0	430	+430
dom7	480	694	+214
m7b5	240	134	-106

quality	instructed	performed	difference
sus	0	132	+132
total	4320	4320	0

The segment total is unchanged, and the choice is therefore not one between more and less data, but a **relabelling of the same recordings**.

The `min` and `min7` rows must be read together: **five hundred segments written as `m` were performed as `m7`**. Training on the `instructed` annotation teaches the model to classify a voicing containing a minor seventh as a plain minor chord. This error was subsequently observed in the application, where the chord `Gm7` was recognised as `Gm`.

Furthermore, the `instructed` annotation contains **not a single** instance of the `maj7` and `min7` classes. Until the switch was made, both classes originated exclusively from the two renders of the synthetic dataset, that is from a single instrument processed through a single amplifier. This produced a figure of 100% on the validation set (the same instrument on both sides of the split) together with an absence of robustness on real material.

Switching to the `performed` annotation moved the `Exact` figure from **82.3% to 92.4%**. Root accuracy was unaffected: the two annotations differ with respect to the root in **0 out of 43,056** comparisons.

4.3. The split must follow the source

Randomly shuffling the list of chord segments and splitting it 94/6 places adjacent bars of **the same recording** on both sides of the split — with the same instrument, room, microphone and take, and frequently with the same chord recurring one bar later.

In the synthetic dataset the dependency is stronger still: the `clean` and `eob` renders of one block constitute the same performance processed through a different amplifier, and were assigned to the training and validation sets independently.

The solution adopted is grouping by source: the entire file for GuitarSet and the entire block (both renders) for the synthetic dataset.

All validation figures fall following this change. This does not constitute a regression of the model, but the removal of an inflation that invalidated earlier conclusions regarding generalisation. The value `root_acc = 98%` obtained in run `take1` was to a large extent an artefact: with both annotations the same segment occurred twice, so an identical window entered both the training and the validation set.

4.4. Pitch targets derived from `note_midi`

The chord annotation describes the **intended harmony** over a span of several seconds. The training window covers 0.77 s and frequently does not contain the labelled seventh at all. The model was therefore penalised for failing to predict a note absent from the signal — seventh recall on GuitarSet stood at 32%.

The hexaphonic pickup provides `note_midi` annotations describing the actual performance on each string separately. Deriving the pitch targets from these raised seventh recall from **32% to 96%**.

The threshold adopted: a note must sound for at least 25% of the window (`NOTE_MIN_COVER`) in order to be included in the target.

5. Architecture

5.1. Two signal paths in one model

Take7 is a single self-contained `best_model_v2_take7.onnx` with two independent inputs and four outputs. The application extracts the required branch into each worker's memory at start-up. It writes no derived model files. Selecting just one output of the full graph did not skip the other branch in profiling; the in-memory separation keeps chord computation out of the 16 ms onset cycle.

```
mono guitar audio
├─ chord DSP: 16 kHz → FFT 8192 → sparse CQT/chroma/bass
│   └─ features [1,48,168] → root, quality, sounding pitches
└─ onset DSP: 16 kHz → Hann spectra 1024/2048
    └─ short_features [1,770,35] → per-pitch Rise events
```

The chord path retains the take6 features: 144 log-normalised CQT bins, 12 chroma values and 12 bass-energy values. The sparse kernel has 24 bins per octave across six octaves from C1. Its context is 48 frames at a 256-sample hop, approximately 0.77 s; inference runs every 40 ms.

The onset path uses 64 ms and 128 ms spectral windows, frequencies through 4 kHz, and `log1p` amplitude compression without per-recording normalisation. Features are rounded to the same float16 precision used by the training cache. Causal resampling, left padding and frame timestamps agree with the trainer. Inference runs every 256 samples, or 16 ms, using 34 past feature frames plus the current one. History is retained continuously; no future audio is required. The update interval is not an assertion of 16 ms end-to-end detection latency.

5.2. Chord base and Rise network

The chord branch preserves the CNN with Squeeze-and-Excitation and the four-layer Transformer encoder of take6. Its CLS token supplies root, quality and sounding-pitch predictions. The previous `fc_onset` head is removed.

Rise uses a 770-to-96 raw-feature projection and a second projection of the positive spectral change, followed by four causal residual convolutions with dilations 1, 2, 4 and 8, and 12 output logits. In amplitude space, its additional input is the positive difference between the current spectrum and the mean of the preceding four frames. The current frame is excluded from that baseline. The difference is compressed back to the feature scale inside the ONNX graph.

Input/output	Shape at live inference	Meaning
<code>features</code>	<code>[1,48,168]</code>	chord CQT context
<code>short_features</code>	<code>[1,770,35]</code>	causal onset context
<code>root_logits</code>	<code>[1,13]</code>	12 roots plus Noise
<code>quality_logits</code>	<code>[1,11]</code>	chord family, single note or noise
<code>pitch_logits</code>	<code>[1,12]</code>	sounding pitch classes, after sigmoid
<code>onset_logits</code>	<code>[1,12,35]</code>	attack probabilities after sigmoid; use the last frame

Metadata records the onset threshold, feature specification, history length and source-model hashes. The application rejects an incompatible contract. The onset threshold is separate from the sounding-note threshold exposed in the UI.

5.3. Why replace the previous onset head?

The training task is to recognise a fresh attack of a particular pitch class while other notes may still be ringing. A sustained note can match the next chord's requested interval without having been played again. A general loudness attack is not enough to say which note was newly struck.

The old head already examined CQT/chroma growth and changes in encoder tokens. Its input, however, came from the 512 ms FFT window of the chord path and its execution followed chord inference. Rise receives shorter spectral windows, learns from both the raw spectrum and its fresh growth, and runs independently. It supports simultaneous attacks of several pitch classes without requiring silence between them.

In guitar practice, the Rise path was reported to substantially reduce credits carried over between chords. Earlier iterations also required some quiet notes to be repeated. The current integration received positive practice feedback. These observations are distinct from controlled accuracy measurements; the historical onset F1 in section 7 cannot be compared directly with Rise event metrics. Packaging both branches into one file does not itself improve accuracy.

5.4. Division of tasks

Root and quality identify chords; pitch describes what is sounding. Rise supplies new attacks for note practice when **Credit only what was struck** is enabled. The judge consumes timestamped events, while order and single-note options still determine how the exercise progresses. With that option disabled, the existing sounding-note rules remain available. Chord recognition has its own latch.

The quality head remains necessary: in an earlier comparison on the same checkpoint, it achieved 80.5% accuracy, compared with 66.0% for templates based on predicted pitches and 59.2% for templates based on the true pitch set. Those are historical chord-head measurements, not Rise metrics.

6. Training

6.1. Take7 training and recovery

`dist/model_trainer.py` is the supported standalone Kaggle script, generated from the chord and onset source modules. The default `auto` mode resumes its own run, otherwise reuses a take6 chord checkpoint and trains only Rise. If no base exists, it also trains the chord branch. `onset_only` requires a ready base; `full` ignores take6 but can resume its own run. A new run tag starts a separate experiment. `USE_HF=False` supports training without Hugging Face history.

Stage	Current role
Chord phases 1–2	Main training and pitch threshold selection, when a base is needed
Chord phase 3	Optional frozen-encoder fine-tuning; disabled by default
Onset stage	Train the separate Rise branch, with chord weights frozen
Export	Merge the selected branches into one ONNX and verify output parity

Take6 chord snapshots do not contain Rise weights. Rise therefore starts fresh unless an `INITIAL_ONSET` PyTorch checkpoint is supplied; an existing take7 resume snapshot takes precedence. The retired `fc_onset` weights are not reused. The onset optimizer never receives the chord parameters.

The default onset run has 12 epochs. A last-epoch snapshot stores model, optimizer, RNG state, history and the best checkpoint together. Data identities, splits, audio/feature hashes and annotations must match when resuming. A mismatch stops the run; it does not silently restart it. In particular, restoring a file path is not permission to change the underlying training data.

For a completed earlier two-file run, `MODE="export_only"` combines its chord ONNX, onset ONNX and saved threshold without training, GPU or feature extraction. The final artifact is `best_model_v2_take7.onnx`; `_chords.onnx` is an intermediate, not the complete application model. Preserve the training summary and checkpoints. The exporter checks both branches against their original ONNX outputs at three batch/context shapes before promoting the combined model.

The following notes explain two decisions retained from chord-model development.

Phase 2 scanned thresholds over the range 0.30–0.70, optimising the `exact` figure. That figure is the conjunction of `argmax(root)` and `argmax(quality)`, and therefore took an identical value across all 41 thresholds, making the selection arbitrary. Sorting is now performed by the F1 of the pitch head, on which the threshold genuinely bears.

Phase 3 was disabled after its effects were measured over three consecutive runs:

```
take2, 40 epochs: pitch_f1 0.9318 → 0.9326 (+0.0008), exact 0.5455 → 0.5445
take3,  4 epochs: F1 0.933 → 0.931,          exact 54.6% unchanged
```

The encoder remains frozen and only the heads are trained, at a learning rate of 1e-5. These trials did not yield a useful gain, while the phase cost approximately 1.5 hours of computation.

6.2. Chord losses and masking

- **root** — CrossEntropy with label smoothing of 0.05,
- **quality** — CrossEntropy with smoothing, sampler weighted by class,
- **pitch** — Focal BCE ($\gamma = 2.0$, `pos_weight` 2.5), auxiliary weight 0.7.

Two masking mechanisms were applied, both justified by measurement.

`MASK_ROOT_WHEN_SILENT` — the root loss is computed exclusively on windows in which the root genuinely sounds. Training the root on windows in which it is absent does not result in learned perception, but in memorisation of the GuitarSet progressions, while the shared encoder receives a gradient in conflict with the pitch target.

`GUITARSET_SOLO_MODE = "mask_chord"` — the root and quality receive no gradient from solo recordings; the pitch head receives it unchanged.

6.3. Chord augmentation

- **pitch shift** by $\pm N$ semitones. An implementation detail of consequence: the CQT and the bass energy are shifted **with zero fill**, whereas the chroma is shifted **cyclically**. The chroma is circular by definition; the CQT is not, and wrapping the bass band around to the top of the range would introduce notes absent from the signal.
- **time and frequency masking** (SpecAugment),
- **spectral tilt and noise** — simulation of varied signal chains.

6.4. Chord energy gate

The parameter `ENERGY_KEEP_FRAC = 0.55` rejects windows whose energy falls below 55% of the segment peak. The justification: during the decay phase the seventh, being the quietest component of the voicing, is the first to disappear, while the label remains unchanged. In the absence of the gate this would systematically train the collapse `m7` $\rightarrow$ `m`.

6.5. Chord metrics

Chord metrics are computed **exclusively on windows in which the label describes the signal**, with solo windows excluded. They are additionally reported separately for windows with and without an audible root, since the combined figure conflates two distinct populations.

Selection of the best checkpoint proceeds by the figure `composite = (root_audible + qual + exact) / 3`. Use of the combined `root_acc` would favour a model that reproduces progressions effectively over one that analyses the signal correctly.

The `TRAIN` diagnostic check, performed every 5 epochs, computes metrics on the training data without augmentation. It addresses the question of whether the model is capable of reproducing its own training data. A negative answer identifies the features or the labels as the source of the constraint rather than generalisation, and indicates that increasing the number of epochs would serve no purpose.

6.6. Rise data, targets and evaluation

Onset training uses synthetic plucks and GuitarSet note events from both solo and comp recordings. String-separated JAMS annotations provide labels; the network input is ordinary mono audio, not the six pickup channels. The pipeline selects the available mic or mono mix variant and records that choice. GuitarSet note starts describe played notes, not independently verified picking technique.

GuitarSet players 00–03 form training, 04 validation and 05 test. Synthetic excitation groups are separated across the three splits. Their eight cases cover held roots, added thirds/fifths/octaves, repeated plucks and held or repeated triads. The tuned `onset-ks-v2` generator uses fractional delay so rendered pitch agrees with its label; deterministic annotations alone would not ensure that.

The default recipe uses BCE with positive weight 4 and gain augmentation of ± 6 dB. Targets mark a pitch-class onset over 96 ms. Simultaneous starts of the same pitch class remain a limitation of this 12-class representation; their overlap counts are reported. Earlier paired/ringing loss experiments remain available in the sources but are not enabled in the take7 recipe.

Checkpoint selection uses validation event metrics; the final threshold sweep uses the exported ONNX predictions. Precision, recall, F1, extra events per minute and audio-time latency are reported separately for domains and cases. The held-out test is evaluated at the validation-selected threshold. A detector event score is not an application-credit score: judging, noise gating and UI scheduling are separate. Previously inspected guitar recordings are regression material, not an unseen test set.

See [training and export instructions](#) and the [tool index](#) for reproducible commands and artifacts.

7. Results and verification

Historical model v2_take6, validated with a split by source and with solo windows excluded:

metric	value
root accuracy	98.1%
pitch F1	0.909
exact match (root and quality)	92.4%
take6 onset F1 (historical)	0.812

The first three figures are identical in the three-head and four-head files: the onset head was trained with the rest of the network frozen. The fourth figure is taken at the threshold maximising F1 on the validation split.

Accuracy by quality at the best checkpoint: dom7 97%, min7 93%, min 92%, sus 91%, maj 89%, maj7 89%; the classes m7b5, dim7 and aug above 97%.

7.1. Course of the work

run	change	Exact
take1–take3	various, on a split subject to leakage	not comparable
take4	split by source — reference point	44.8%
take5	masking of solo recordings	82.3%
take6	performed annotations	92.4%

7.2. Limiting factor

The difference between the training and validation sets with respect to quality amounts to **6.5 percentage points** (99.2% against 92.7%). The model reproduces the training data. This corresponds to the profile of a constraint imposed by **generalisation** rather than by the capacity of the architecture.

Practical conclusion: neither an increase in the number of epochs nor an increase in model size will yield improvement. Improvement will follow from an increase in the quantity of varied material from a real instrument.

7.3. Take7 integration checks

The combined-file integration was tested with a control merge of existing take6 chord weights and existing Rise weights. The chord/pitch outputs matched their original model on four input windows. On the full AtoA recording, 5682 frames and 50 detected events matched the separate Rise model, with zero probability difference, both with and without optional weak-response rescue.

Profiler traces showed 32 executed nodes in the onset session and 437 in the chord session, with no nodes from the other branch. Runtime tests also exercised event consumption, boundaries, input restarts and polyphonic crediting. These checks establish preservation of the original branches and their scheduling; they are not a new accuracy evaluation of subsequently trained take7 weights. The current take7 file passes the application's `--check` inference check.

New training metrics belong to that run's `training_summary_v2_take7.json`, including the selected checkpoint, threshold and model hashes. The older onset F1 above uses a different evaluation procedure and is not a take7 result.

8. The application

Recognition accuracy is not equivalent to the usefulness of a trainer. Three matters proved to carry weight comparable to changes in the model.

8.1. Requirement for a full context window

The trainer determined windows **exclusively within** the sustaining chord (`range(start, end - 48)`). Following a strike of the strings, the application buffer contains partial silence for 0.77 s; taking the FFT window into account (8192 samples, that is 512 ms), the oldest frame describes signal from as much as 1.3 s earlier. This constitutes an input outside the training distribution.

The observed symptom: seventh chords were recognised only during the sustain phase, that is at the first moment at which the window becomes entirely filled with the chord.

The solution adopted was a single threshold: the application issued no query to the model until the window was 90% filled with signal. That threshold is correct for the chord NAME and wrong for everything else. Playing one note at a time never fills a window to nine tenths — 43 frames, that is 688 ms of unbroken sound — so the model was not asked at all, the display froze on the last chord it had, and everything downstream froze with it.

The requirement is now split in two. The model is asked from half a window (measured: at 50–70% fill the pitch head names an isolated note correctly in every frame of the measurement), and its chord name is believed only from nine tenths. Both thresholds are stated as constants at one place in the application, because the gate on the name is applied on the far side of the channel from the thread that asks, and written out twice they were free to drift apart.

8.2. Differing decay rates of chord components

Diagnostic output for a sustained Gm7 chord:

G m7	min7=96%	b7=96	← immediately after the strike
G m7	min7=82%	b7=76	
G m7	min7=52%	b7=52	
G m	min=49%	b7=45	← the seventh has faded, the model reclassifies

The model's classification is correct — the seventh is genuinely no longer present in the current window. A chord does not, however, change its identity in the course of sustaining.

The solution adopted: a quality latch. The mechanism **engages** at a confidence of ≥ 0.60 but **holds its state independently of confidence**. During the decay phase the model reports the reduced quality at a confidence in the region of 94–96%, so a confidence threshold alone would afford no protection. The latch is released by a new attack or by a change of root.

The latch engages only 48 frames after the attack. Before that point the window still contains the tail of the **preceding** chord, which would result in an incorrect name being latched.

Attack detection compares the signal level against a slowly moving envelope (EMA) rather than against an absolute threshold, which would depend on playing volume. A refractory period of 0.2 s was additionally applied so that a single strum triggers exactly one attack.

8.3. Measuring time rather than assuming it

The progress counter received the fixed value `dt = 0.040`, whereas the inference thread required 55–90 ms per cycle (inference together with 40 ms of sleep). The counter therefore ran slower than real time: a threshold of 0.6 s was reached after approximately one second, with the value depending on machine load.

Following correction — comprising measurement of real elapsed time, determination of the period by the inference thread from the start of the cycle, narrowing of the voting window from 5 to 3, and reduction of the default threshold to 0.25 s — the transition takes **approximately 0.3 s** in place of 1.2 s.

8.4. Sparse representation of the CQT kernel

The full kernel comprises $4097 \times 144 = 589,968$ weights, concentrated around the centre frequency of each bin. Discarding weights below $1e-4$ of the peak value yields the following results:

threshold	weights retained	max. error relative to peak
1e-5	21.9%	0.006%
1e-4	6.9%	0.033%
1e-3	2.3%	0.352%

The error was determined on three spectra: white, pink, and a guitar-like harmonic series. The CQT is followed by log normalisation over an 80 dB range, so a figure of 0.03% remains orders of magnitude below the resolution of the feature.

The size of the weights file falls from **28 MB to 2 MB**, and the audio path performs approximately **14 times fewer multiplications** per frame. The previous implementation traversed all 4097 FFT bins for each of the 144 CQT bins, discarding zero values only within the loop.

8.5. Feature agreement between the trainer and the application

Discrepancies of this class are particularly difficult to diagnose, since the application remains functional and exhibits only classification errors. Two were identified:

- **chroma mapping.** The file distributed with the application folded bins in the pairs `(0,1)`, `(2,3)`, ..., whereas `librosa.cq_to_chroma` applies the division `(1,2)`, `(3,4)`, Every second bin was assigned to the adjacent class, which corresponds to a chroma blurred by a semitone across half the band.
- **cache key.** The cache file name was derived from the expression `abs(hash(path))`. The Python interpreter randomises the hash seed for strings on every process start, so the cache was never reused between sessions. An SHA-1 digest of the file name is now employed.

The application **rejects** weights in the previous, dense format, reporting this by means of a message.

8.6. User interface

Following a review, the number of controls was reduced from five to four:

control	remarks
Noise gate	in dBFS, with a level meter on the same scale and a threshold marker

control	remarks
Chord confidence	threshold for the chord name (Chords mode)
Note threshold	threshold for a single note (note-based modes)
Hold time	the period for which a correct chord must be sustained

The `Tail` control (set from the interface and read nowhere in the code) and the `In gain` control (whose effect was cancelled by the per-frame normalisation, so that it shifted only the same inequality as the noise gate) were removed.

The `Confidence` control governed two distinct quantities simultaneously, and in the note-based modes was subject to a lower bound of `.max(0.5)`, as a result of which the entire range 0.1–0.5 produced identical behaviour. The functions were separated.

The noise gate previously operated on a linear RMS scale of 0–0.1, which **did not reach the noise level of a laptop microphone** (RMS 0.05–0.15 after gain). A decibel scale of –72...0 dBFS provides resolution across the required range together with coverage to full scale.

The panel has since grown past what one column can hold and is divided into four tabs — the input and the gate, how strictly what is played is judged, what is to be played, and what the window shows. The third of these holds only what belongs to the mode on screen: a song has nothing to say in Formulas and a formula nothing in Chords, so it is a different tab in each of them. Only one tab is drawn at a time, so there is correspondingly less to redraw while playing.

8.7. Diagnostic mode

```
SOLITITO_DEBUG=1 ./solitito
```

The mode prints, on each prediction, the three strongest qualities together with the pitch vector expressed as **intervals relative to the recognised root**:

```
G m7 | min7=97% sus=0% maj=0% | R96# b25 28 b382# 37 44 b56 594# b616 69 b797# 74
```

The tool distinguishes the case in which the model fails to detect the seventh from the case in which it detects it but disregards it in classification. Both symptoms are indistinguishable at the level of the chord name and call for opposite corrective actions. The `Gm7` case was resolved by these means without retraining.

8.8. Single notes are not a question the model can answer

This section records the earlier CQT solution and its measurements. Take7 retains that path, but practice requiring a fresh strike uses the independent Rise events described in section 8.12 instead of waiting for the chord window.

The model is asked about 48 frames — 0.77 s — and answers about all of it. That is correct for a chord held under the fingers and wrong for a scale, where notes follow one another faster than the window empties.

Measured with `--probe` on a scale at 0.6 s per note, the rule then in force (the target class above threshold and within 10% of the loudest) credited:

	the note being played	the note before it
the model's pitch head	7% of windows	79%
a single CQT frame	57%	43%

The model is not at fault: on isolated sustained notes it places 0.96–0.99 on the correct pitch class, and on the scale it is reporting both notes because both were in the window. The older of the two wins on level, having had more of the window to itself.

The solution adopted: the note-based modes ask a second question of one CQT frame, which carries no memory. A harmonic sum over the log-magnitude bins — the harmonic product spectrum, the axis being logarithmic — yields the pitch class sounding at that moment. It never named a class that was not played.

By default the estimate only **adds** a route to a pass, since overruling the model would forfeit the property that distinguishes this trainer from a monophonic one: the pitch head is polyphonic, so a strummed chord walks its intervals one by one. The **Play the notes one at a time** option makes the estimate the authority, and additionally requires a fresh attack before a repeated note counts a second time.

The residual latency is the 8192-sample FFT window, half a second wide, which is why notes shorter than approximately 0.4 s remain difficult. A shorter-window estimator in the time domain (autocorrelation) is the remaining avenue.

8.9. Input selection, and what a device list omits

A Windows machine reported no signal until the sample rate was changed by hand, which established that the sample format returned by the backend must not be discarded, and that the choice of device belongs to the user rather than to the operating system default.

The device list carries one property that is not obvious: **a card can be opened once**. Whatever holds it — a sound server, another application, or this application's own stream — removes it from the enumeration entirely. Three consequences follow, each of which was first observed as a defect:

- a list built after the stream opens is missing the card being recorded from,
- under PipeWire, which claims the hardware, only the four server names remain,
- the device being recorded from must be exempted from any "unavailable" marking, since its absence from the scan is the evidence that it is working.

The noise gate is stored per device. An interface and a laptop microphone sit tens of decibels apart, and a threshold that has to be found again after every switch is not a setting.

8.10. Independent inference schedules

Chord inference runs every 40 ms, while Rise processes each 16 ms audio hop in its own worker. `--bench` measures the chord branch; it is not a measurement of the complete onset/crediting path. Splitting the combined ONNX in memory at load time avoids running the chord encoder on onset-only hops.

Earlier measurements of 39 ms on Linux and 61 ms on Windows concerned chord inference on the reference machine. They are historical measurements, not the latency of take7's Rise branch. CPU load must also be compared on the same basis: one fully used core appears as 12.5% when a meter reports over eight cores.

8.11. Formulas, and a rule stricter than the one the note modes use

The following comparison records the earlier CQT/take6 design. In the current build, enabling **Credit only what was struck** routes Formulas through Rise as well; with the option disabled, the existing sounding-note policy applies.

The application draws a set of intervals over a root — every subset of the twelve chromatic functions containing the root, 2048 in all — and the exercise is to find them on the neck and play them in any order. A function once credited stays credited for the lap, which changes what a false credit costs: in the other modes a wrong reading delays the exercise, here it removes a function from it permanently.

The rule was therefore measured rather than assumed. Over 49 notes of a real recording (`dist/latency_stats.py`):

rule	false credits	notes missed
all four paths at once, as the note modes use	110	0
the single-frame CQT estimate alone	33	0
the same, gated on the onset head	15	4

Of the 110, ninety-nine came from the model's pitch head — which answers "what is sounding", and a string ringing on or resonating in sympathy is sounding without having been played. That version therefore used the single-frame estimate alone, with a vote of four of the last five audio frames. The old onset gate was not adopted because it missed four played notes in this comparison.

The same mode also plants a formula on a chord: its root is placed on one of the chord's twelve degrees, and how much of that chord the set then covers is counted — every tone of the chord except its root and its perfect fifth, since only those establish which chord it is. This is arithmetic over two twelve-bit masks and is exact, which is what makes it worth showing on screen beside the functions.

8.12. Events and exercise boundaries

A sounding pitch and a fresh attack carry different evidence. With Rise and **Credit only what was struck** enabled, the application consumes events tagged with a pitch class, audio timestamp, frame identity and input generation. Already consumed, expired or pre-boundary events cannot credit a new target. Pause, exercise transitions and input restarts clear pending evidence; audio queue discontinuities reset the detector context rather than pretending the stream was continuous.

A polyphonic attack may supply several pitch-class events for one exercise. Order and single-note policies decide which of those events the exercise can use. The detector's peak-relative rearming is retained across exercise changes; changing the target does not itself constitute a new attack.

The optional `SOLITITO_ONSET_RESCUE=1` path confirms a weaker response through pitch-specific spectral growth and one additional frame. Its timestamp remains that of the candidate, so confirmation does not move an earlier attack past a round boundary. Strong responses do not incur this additional wait. Rescue is disabled by default and is independent of model selection and audio recording.

Normal startup automatically selects `best_model_v2_take7.onnx` when present. `SOLITITO_MODEL` overrides the primary path; `SOLITITO_ONSET_MODEL` overrides the onset source. For comparison, take6 plus `SOLITITO_ONSET=legacy` remains available. The trace/recording script is optional; normal practice needs neither a trace nor a WAV recording. See [running the app](#).

9. Hypotheses refuted by measurement

The onset-related entries in this table concern the earlier take6/CQT path, not the current Rise branch.

This chapter documents cases in which measurement refuted a previously held assumption.

hypothesis	measurement result
A normalisation discrepancy (per frame against global) constitutes the principal constraint	difference below 1 pp; the <code>InstanceNorm2d</code> layer compensates for it
A harmonic barrier — the third harmonic of b3 coincides with b7, rendering min7 and min indistinguishable	the measurement had been performed on segments carrying random labels
The model does not reproduce the training data (underfitting)	TRAIN check: 83.7% against 63.1% on validation, hence overfitting
Template matching will outperform the quality head ($B \approx 75\%$ against $A = 63\%$)	$B = 61.0\%$
The 64% root ceiling follows from root-less jazz voicings	an artefact of the solo recordings; 97% in accompaniment
Masking the root will unblock quality beyond 73%	quality remained at 72%
The chroma in the distributed file is one-hot and therefore defective	<code>cq_to_chroma</code> at 24 bins per octave likewise assigns one weight per bin; the discrepancy concerned a shift
Without the <code>ORT_DYLIB_PATH</code> variable the binary will use the system library	<code>RUNPATH=\$ORIGIN</code> from the <code>.cargo/config.toml</code> file already addressed this
The model has become worse at single notes	on isolated notes it places 0.96–0.99 on the correct class; on a scale its 0.77 s window credits the note before the one being played, in 79% of windows
A diminished seventh named from another of its notes is a different chord	it is the same four notes: C, Eb, Gb and A dim7 differ only in which one the model calls the root, and that follows the voicing rather than what was played
The onset head will make the better gate — it is the fastest answer available	on a recording it looked so: 202 ms against 676 ms. Applied live it refused far more than it caught, and on the crediting rule it traded 18 false credits for 4 notes missed
A third credited while the root was played is the fifth harmonic of that root	<code>--probe</code> over 364 windows: the false credits fall on +10 and +11 semitones, that is on the PREVIOUS note still inside the window, not on a harmonic
A single fresh answer from the onset head is too short a window to catch a strike	at a threshold of 0.02 the answer stays above the gate for a median of one second after the attack, and not one of 47 notes was left without a frame carrying it — the sixteen-frame memory kept for the purpose was removed

The pattern is unambiguous: **measurement results held consistently, whereas predictions formulated prior to measurement proved wrong in a systematic manner.** This justifies the adopted methodology based on probes.

10. Design decisions

10.1. Solutions adopted

The generator writes labels directly. No processing step reconstructs information from the signal.

Verification of labels by an independent script. A measure redundant with respect to the generator, applied deliberately.

Splitting the dataset by source. It lowers the reported figures by more than ten percentage points and is justified.

Four outputs, two independent branches. Root, quality and pitch preserve the chord base. Rise supplies timestamped attacks for note practice requiring fresh strikes. Both branches are distributed in one ONNX, with separate inputs and inference schedules.

Two thresholds on the context window rather than one. The model is asked from half a window and its chord name believed from nine tenths — a single threshold cannot serve both a held chord and a single note.

Nothing heard before a boundary survives it. A chord change, a lap and a mode switch each discard the model's last answer, because that answer is about what came before them.

A sparse representation of the CQT kernel. The benefit is twofold: the size of the weights file and the processing time in the audio thread.

Rejection of incompatible weights by the application. Silent acceptance would result in a program that is functional but misclassifies.

Compiled-in strings, without the gettext library. With several dozen strings, a system dependency and `.mo` catalogues incur a cost exceeding the benefit.

10.2. Solutions rejected

Deriving quality from the pitch vector — measured as 21 percentage points inferior to the quality head.

Temporal aggregation as a means of improving quality — on a controlled population it yields approximately +1 pp. The model's errors are correlated in time: the model exhibits no vacillation between windows, but indicates the same incorrect answer consistently and with high confidence.

A bar-based mode (a score advancing in tempo, with assessment in place of a gate) — implemented and subsequently withdrawn. It was determined that the trainer is to respond dynamically.

Training phase 3 — measured as yielding no improvement across three runs.

10.3. Further validation

- **Independent recordings from target instruments and interfaces.** AtoA and practice recordings already inspected are regression material; a controlled comparison of model weights needs additional unseen material.

- **Stable resume and provenance.** Keep the selected model, threshold, data identities and training summary together. A changed-data check must not be bypassed to resume an unrelated run.
 - **Release packaging.** The 0.5.7 workflow bundles the combined take7 artifact, verifies its SHA-256 hash and runs both model branches with `--check` in the assembled Linux and Windows packages before publication.
-

11. Conclusion

The take6 work resulted in a model achieving 92.4% exact matches on validation determined with a split by source, released as distribution packages for two platforms.

The principal gain in accuracy followed not from changes to the architecture, but from four findings concerning the data:

1. half of the GuitarSet dataset consists of improvisation labelled with accompaniment chords,
2. the `instructed` annotation contains no sevenths and misclassifies five hundred segments,
3. splitting the dataset at the segment level introduces leakage,
4. pitch targets must be derived from the actual performance rather than from the score.

These four changes moved the `Exact` figure from 44.8% to 92.4%. None of them concerned the structure of the network.

Take7 adds a separate development step: short causal onset features, a Rise network dedicated to fresh attacks, and timestamped events in the application. The chord base can be reused without retraining, or the entire model can be trained from scratch. A checked export delivers both branches in one ONNX while preserving their independent runtime schedules. This extends the application from identifying sounding material to judging newly played notes.

Updated 4 October 2026: take7, application version 0.5.7. Repository: <https://github.com/greblus/solitito>